Ambient GitHub PR Status Display

Final Presentation

Devyn Ferman

CSC 494: AI-Driven IoT System Development

What I Set Out to Build

Problem: Constantly switching to GitHub to check PR status breaks focus.

Solution: A physical, always-on display that passively surfaces the right info. No tab switching required.

- Swift CLI polls GitHub on a configurable timer
- Sends status to an Arduino-driven 16x2 LCD over serial
- Priority logic: review requests → drafts → open PRs → all clear

Progress Since S1P

At S1P:

- Hardware powered, LCD showing default squares (not yet text)
- Swift CLI skeleton running, first GitHub API call working

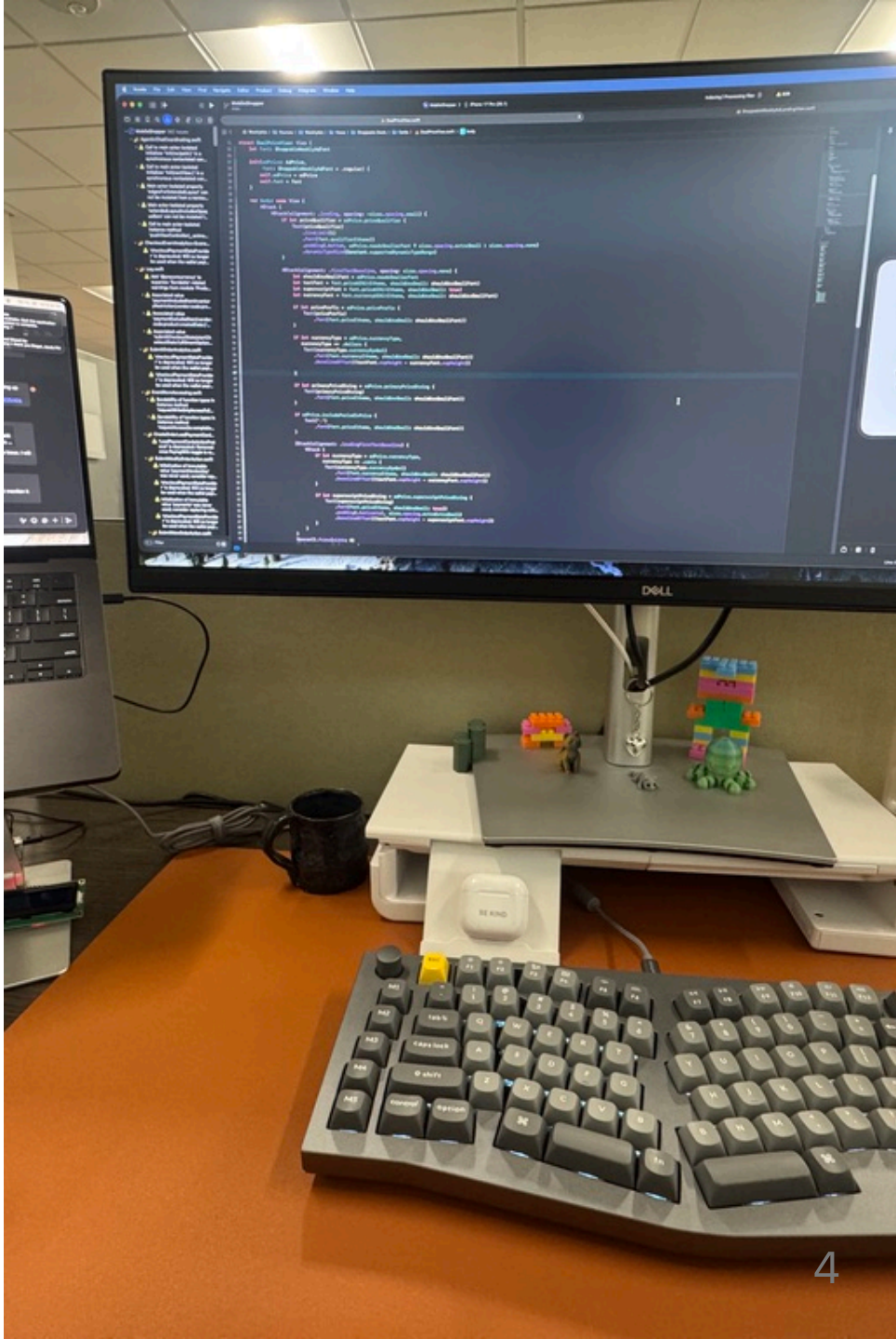
Since S1P:

- LCD displaying text via I2C firmware
- POSIX serial connection implemented in Swift
- Full GitHub polling: authored PRs and review-requested PRs
- Priority-based display logic and change detection
- Error recovery: API failures surface on the LCD and the loop keeps going

It Works!

Not only did I manage to get everything up and running ahead of time, I started actually using it as part of my workflow! All I needed to do was plug it in and run a single command to have my passive notifications running.

```
./start-watcher.zsh <github-username> <GitHub Auth Token> [Options]
```



Issues I Faced

Hardware: Shared Ground

The LCD wouldn't respond at all. Once I understood that I2C requires all devices to share a common ground reference, the fix was a single wire.

Software: No Serial Abstraction in Swift

Swift doesn't have a built-in serial port API. I had to drop into Darwin's C layer using POSIX `termios`, configuring baud rate, stop bits, and parity by hand.

Architecture: The Original Plan Was Too Complex

The initial design had an ESP32 fetching a JSON status file over WiFi. Talking it through with AI made clear that a local Swift process sending messages directly to the Arduino over serial

How AI Helped Me Solve Them

Shared ground: AI explained why I2C requires a shared ground reference and what the symptom of a missing one looks like, which matched exactly what I was seeing.

POSIX serial: AI walked me through the `termios` configuration pattern step by step. It's a C API from the 1980s with no modern documentation, and I wouldn't have gotten through it quickly (or at all) on my own!

Architectural pivot: AI helped me compare the two approaches side by side. The tradeoffs were obvious once I listed them out, and the simpler path was also the better fit for how I actually use the device.

What I Learned from AI

AI was most useful as a **thinking partner**, not a code generator.

- It helped me ask better questions ("what do I actually need?" vs. "how do I implement X?")
- It shortened the feedback loop on unfamiliar domains: hardware wiring, POSIX APIs, GitHub Search API behavior
- It caught design problems early, before I spent time building the wrong thing

The key was treating AI output as something to understand and verify, not something to copy and run.

Does It Match the Plan?

Original Plan	What Shipped
ESP32 WiFi polling	Swift CLI on Mac
JSON status file	Direct serial messages
Always-on network device	Session-based background process

The scope changed, but the problem and the goal didn't.
The final result is simpler, more reliable, and a better fit for how I actually work.

Q & A

GitHub: https://github.com/DevynFerman/school-projects

Canvas:
https://nku.instructure.com/courses/88152/pages/individual-progress-devyn-ferman